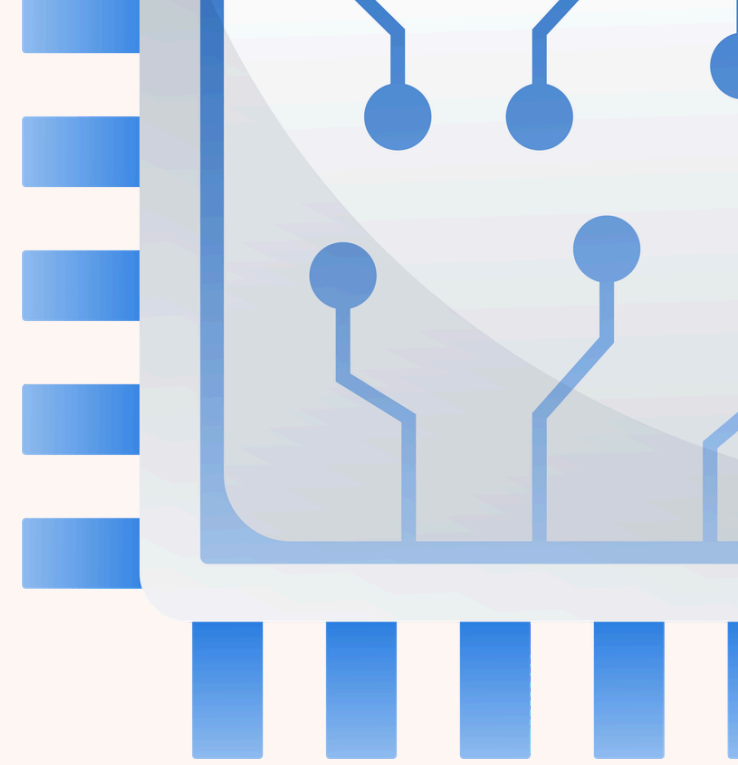
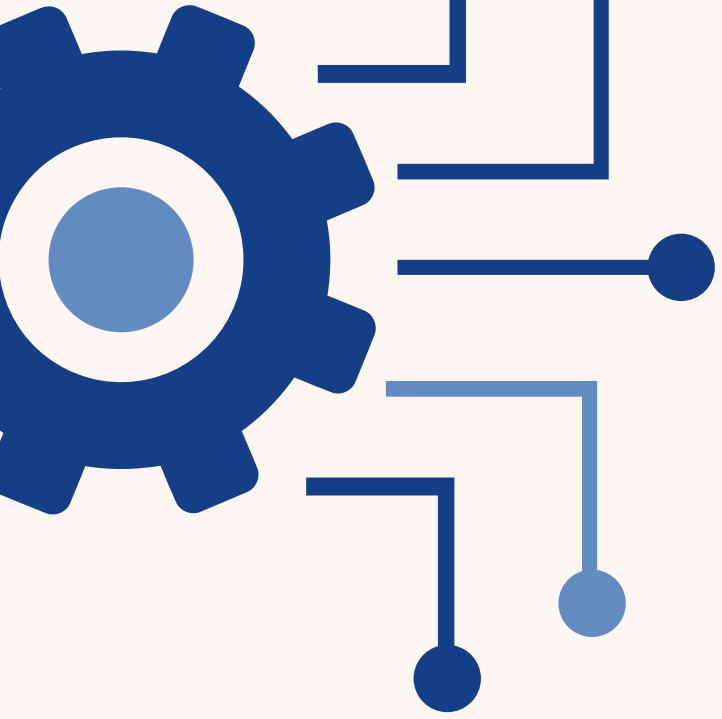
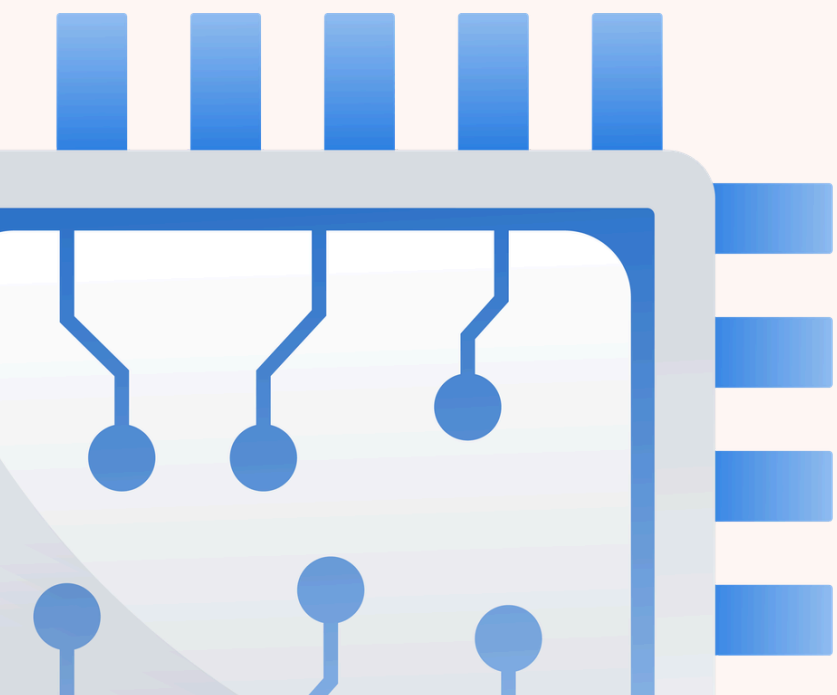




PROYECTO FINAL

Integrantes

Rodrigo Villavicencio
Fabrizio Rodriguez





SEGMENT TREE

Un Segment Tree o árbol de segmentos es una estructura de datos que divide un arreglo en varios rangos o segmentos. Cada nodo del árbol guarda información resumida de una parte del arreglo, como la suma, el mínimo o el máximo.

Esta estructura permite responder consultas sobre rangos de manera rápida, sin necesidad de recorrer todos los elementos uno por uno. Por eso es útil cuando se trabaja con muchos datos y se necesitan varias consultas eficientes.

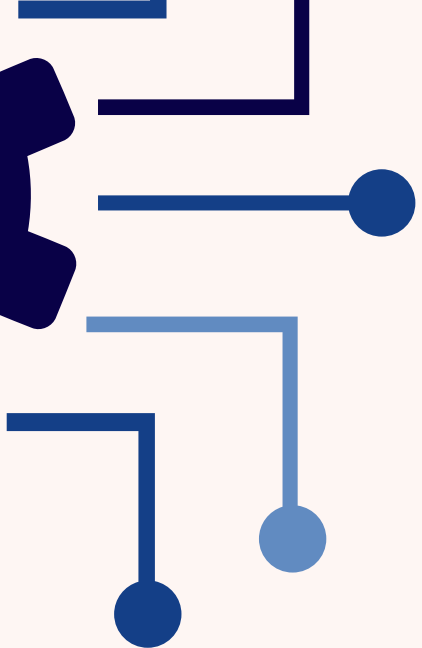


¿PARA QUE SIRVE?

El Segment Tree sirve para realizar consultas rápidas sobre un rango de datos dentro de un arreglo, sin tener que revisar cada elemento uno por uno.

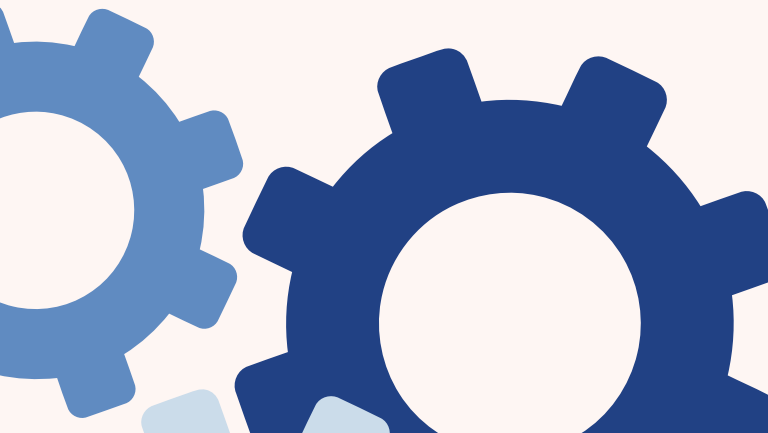
Permite resolver operaciones como:

- Calcular la suma de valores en un intervalo.
- Encontrar el mínimo dentro de un rango.
- Encontrar el máximo dentro de un rango.
- Actualizar valores de forma eficiente.



USO EN EL PROYECTO

En este proyecto, el Segment Tree con lazy propagation se usará como una estructura de indexación para manejar datos por rangos.



USO EN EL PROYECTO

Obtener la suma de un intervalo

```
func (st *SegmentTree) QuerySum(l, r int) float64 {
    return st.querySum(1, 0, st.N-1, l, r)
}

func (st *SegmentTree) querySum(node, left, right, qLeft, qRight int) float64 {
    if qRight < left || qLeft > right {
        return 0
    }
    if qLeft <= left && right <= qRight {
        return st.Tree[node].Sum
    }

    st.push(node, left, right)
    mid := (left + right) / 2

    return st.querySum(node*2, left, mid, qLeft, qRight) +
        st.querySum(node*2+1, mid+1, right, qLeft, qRight)
}
```

Encontrar el valor mínimo en un rango

```
func (st *SegmentTree) QueryMin(l, r int) float64 {
    return st.queryMin(1, 0, st.N-1, l, r)
}

func (st *SegmentTree) queryMin(node, left, right, qLeft, qRight int) float64 {
    if qRight < left || qLeft > right {
        return math.MaxFloat64
    }
    if qLeft <= left && right <= qRight {
        return st.Tree[node].Min
    }

    st.push(node, left, right)
    mid := (left + right) / 2

    leftMin := st.queryMin(node*2, left, mid, qLeft, qRight)
    rightMin := st.queryMin(node*2+1, mid+1, right, qLeft, qRight)

    return min(leftMin, rightMin)
}
```

Actualizar varios valores en vivo

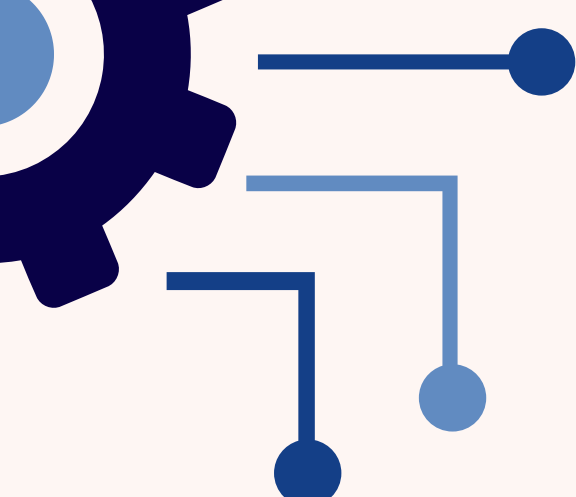
```
func (st *SegmentTree) Update(pos int, value float64) {
    st.update(1, 0, st.N-1, pos, value)
}

func (st *SegmentTree) update(node, left, right, pos int, value float64) {
    if left == right {
        st.Tree[node] = Node{Sum: value, Min: value, Max: value}
        st.Data[pos] = value
        return
    }

    mid := (left + right) / 2
    if pos <= mid {
        st.update(node*2, left, mid, pos, value)
    } else {
        st.update(node*2+1, mid+1, right, pos, value)
    }

    leftNode := st.Tree[node*2]
    rightNode := st.Tree[node*2+1]

    st.Tree[node] = Node{
        Sum: leftNode.Sum + rightNode.Sum,
        Min: min(leftNode.Min, rightNode.Min),
        Max: max(leftNode.Max, rightNode.Max),
    }
}
```



¿QUE ES LAZY PROPAGATION?



La lazy propagation es una técnica que mejora el Segment Tree cuando se necesitan hacer actualizaciones sobre rangos completos.

En lugar de actualizar todos los elementos uno por uno, el árbol guarda una marca pendiente indicando que ese rango debe actualizarse después, solo cuando sea necesario.

Esto permite que las actualizaciones sean mucho más rápidas y eficientes.

